🐣 Human-Embryo *Phase Classifier* — End-to-End Notebook

This notebook trains a deep network that maps a **single time-lapse frame**
to its *morphokinetic phase* (`t2`, `t3`, …, `tB`).

Paths

```
DATA_ROOT           = Path("/content/drive/MyDrive/Msc in AI/AI Applications/Human embryo time-lapse video dataset/embryo_dataset
ANNOT_DIR           = Path("/content/drive/MyDrive/Msc in AI/AI Applications/Human embryo time-lapse video dataset/embryo_dataset
PRESENCE_MODEL_PATH = Path("/content/drive/MyDrive/Msc in AI/AI Applications/Human embryo time-lapse video dataset/models/presenc
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

🔧 Install & Imports

```
# @title 🔧 Install & Imports
# !pip -q install torch torchvision torchaudio scikit-learn torchcam --extra-index-url https://download.pytorch.org/whl/cu118

import random, re, math, json, os, gc
from pathlib import Path
from collections import defaultdict

import numpy as np
import pandas as pd
from PIL import Image
from tqdm.auto import tqdm

import torch, torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as T
import torchvision.transforms.v2 as v2
import torchvision.models as models

from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
```

📁 Paths & Device

```
# @title 📁 Paths & Device
DATA_ROOT           = Path("/content/drive/MyDrive/Msc in AI/AI Applications/Human embryo time-lapse video dataset/embryo_dataset'
ANNOT_DIR           = Path("/content/drive/MyDrive/Msc in AI/AI Applications/Human embryo time-lapse video dataset/embryo_dataset_
PRESENCE_MODEL_PATH = Path("/content/drive/MyDrive/Msc in AI/AI Applications/Human embryo time-lapse video dataset/models/presence

assert DATA_ROOT.is_dir(), f"{DATA_ROOT} not found"
assert ANNOT_DIR.is_dir(), f"{ANNOT_DIR} not found"
assert PRESENCE_MODEL_PATH.is_file(), f"{PRESENCE_MODEL_PATH} not found"

device = "cuda" if torch.cuda.is_available() else "cpu"
print("Running on", device)
```

```
Running on cuda
```

🚫 Embryo Exclusion List

```
# @title 🚫 Embryo Exclusion List
EXCLUDE_EMBRYOS = {
    "BJ492-8",
    "ALR493-6"
}
```

🔍 Load Embryo-Presence Classifier

```
# @title 🔍 Load Embryo-Presence Classifier
presence_model = models.resnet18(weights=None)
presence_model.conv1 = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3, bias=False)
presence_model.fc    = nn.Linear(presence_model.fc.in_features, 2)
presence_model.load_state_dict(torch.load(PRESENCE_MODEL_PATH, map_location=device))
presence_model.to(device).eval()

PRESENCE_SIZE = 224
presence_tfm = T.Compose([
    T.Resize((PRESENCE_SIZE, PRESENCE_SIZE)),
    T.ToTensor(),
    T.Normalize([0.5], [0.5]),
])

@torch.no_grad()
def embryo_prob(pil_img):
    """Return P(embryo present) given a PIL image."""
    x = presence_tfm(pil_img)          # always 224×224
    logits = presence_model(x.unsqueeze(0).to(device))
    return logits.softmax(1)[0, 1].item()
```

⚙️ Helpers

```
# @title ⚙️ Helpers

PHASES = [
    "tPB2", "tPNa", "tPNf",
    "t2", "t3", "t4", "t5", "t6", "t7", "t8", "t9+",
    "tM", "tSB", "tB", "tEB", "tHB"
]
PHASE2IDX = {p: i for i, p in enumerate(PHASES)}
IDX2PHASE = {i: p for p, i in PHASE2IDX.items()}
NUM_CLASSES = len(PHASES)

def extract_frame_idx(name: str):
    m = re.search(r"_RUN(\d+)", name)
    return int(m.group(1)) if m else None

def index_frames(folder: Path):
    """Return {frame_idx: img_path} for every *_RUN###.* file."""
    mapping = {}
    for fp in folder.glob("*_RUN*.*"):
        m = re.search(r"_RUN(\d+)", fp.name)
        if m:
            mapping[int(m.group(1))] = fp
    return mapping

def keep_embryo_frames(paths, empty_thr):
    tensors, kept = [], []
    for p in paths:
        try:
            img = Image.open(p).convert("L")
        except Exception:
            continue
        tensors.append(presence_tfm(img))
        kept.append(p)
    if not tensors:
        return []               # nothing readable
    batch = torch.stack(tensors)
    probs = presence_model(batch).softmax(1)[:,1]
    return [p for p, pr in zip(kept, probs) if pr >= empty_thr]
```

🗄️ PhaseDataset (filters empty frames)

```
# @title 🗄️ PhaseDataset (filters empty frames)

FRAMES_PER_PHASE = 5      # None → keep all

class PhaseDataset(Dataset):
    def __init__(self, embryo_ids, transform,
                 empty_thr=0.5, frames_per_phase=10):
        self.transform = transform
        self.samples   = []           # [(img_path, label_idx), …]

        presence_model.cpu().eval()   # presence filter on CPU

        for eid in tqdm(embryo_ids):
            csv_path = ANNOT_DIR / f"{eid}_phases.csv"
            if not csv_path.is_file():      # skip missing
                continue

            frame2path = index_frames(DATA_ROOT / eid)

            df = pd.read_csv(csv_path, header=None,
                             names=["phase", "start", "end"])

            for phase, start, end in df.itertuples(index=False):
                phase = str(phase).strip()
                if phase not in PHASE2IDX:          # unknown label → skip
                    continue

                # ---- subsample frames ---------------------------------
                if frames_per_phase and (end - start + 1) > frames_per_phase:
                    sel = np.linspace(start, end, frames_per_phase, dtype=int)
                else:
                    sel = range(start, end + 1)

                paths = [frame2path.get(f) for f in sel if frame2path.get(f)]

                # ---- presence filter ----------------------------------
                kept = keep_embryo_frames(paths, empty_thr)

                lbl = PHASE2IDX[phase]
                for p in kept:
                    self.samples.append((p, lbl))

        print(f"Kept {len(self.samples):,} frames")
        self.num_classes = NUM_CLASSES

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, i):
        path, label = self.samples[i]
        img = Image.open(path).convert("L")
        return self.transform(img), torch.tensor(label)
```

🧰 Transforms & Data Split

```
# @title 🧰 Transforms & Data Split
MAX_EMBRYOS = None          # 25 for a quick run

IMG_SIZE = 500
base_pre_tfm = T.Compose([
    T.Resize((IMG_SIZE, IMG_SIZE)),
    T.ToTensor(),
    T.Normalize([0.5], [0.5]),
])

train_aug = v2.Compose([
    v2.RandomHorizontalFlip(),
    v2.RandomVerticalFlip(),
    v2.RandomRotation(20),
    v2.RandomResizedCrop(IMG_SIZE, scale=(0.8,1.0), ratio=(0.9,1.1)),
    v2.GaussianBlur((3,3), sigma=(0.1,2.0)),
])

train_tfm = v2.Compose([*base_pre_tfm.transforms, train_aug])
val_tfm   = base_pre_tfm
```

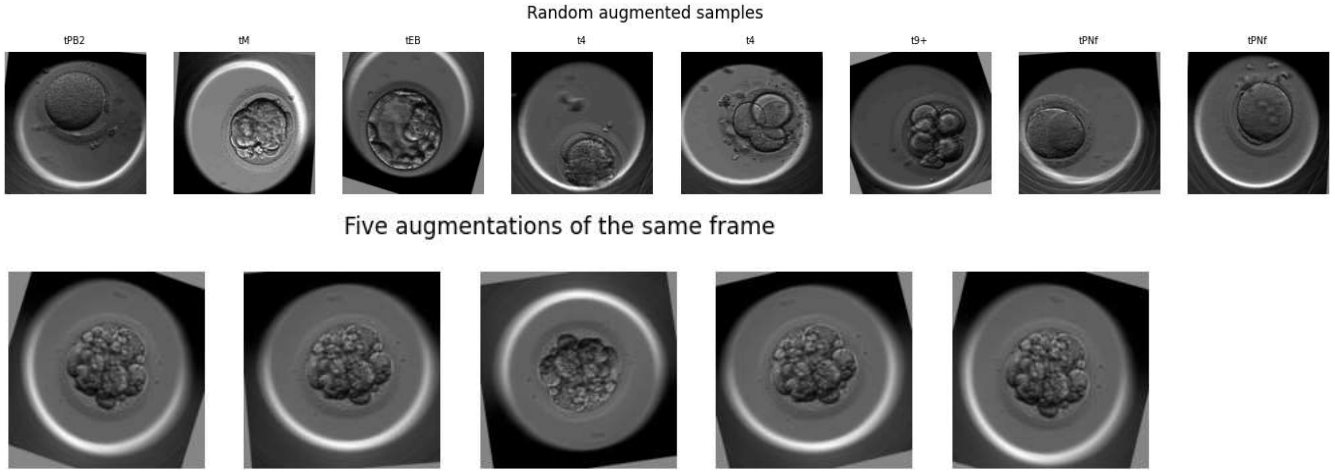
```
# embryo-level split
all_embryos = sorted([p.name for p in DATA_ROOT.iterdir() if p.is_dir()
                      and p.name not in EXCLUDE_EMBRYOS])

random.seed(42)
random.shuffle(all_embryos)

# respect MAX_EMBRYOS
if MAX_EMBRYOS is not None:
    assert MAX_EMBRYOS >= 3, "Need ≥3 embryos to form train/val/test"
    all_embryos = all_embryos[:MAX_EMBRYOS]

n_val = max(1, int(0.10*len(all_embryos)))
n_test = max(1, int(0.10*len(all_embryos)))
val_embryos = all_embryos[:n_val]
test_embryos = all_embryos[n_val:n_val+n_test]
train_embryos = all_embryos[n_val+n_test:]

print(f"Embryos → train {len(train_embryos)} | val {len(val_embryos)} | test {len(test_embryos)}")

train_ds = PhaseDataset(train_embryos, transform=train_tfm, frames_per_phase=FRAMES_PER_PHASE)
val_ds   = PhaseDataset(val_embryos,   transform=val_tfm, frames_per_phase=FRAMES_PER_PHASE)
test_ds  = PhaseDataset(test_embryos,  transform=val_tfm, frames_per_phase=FRAMES_PER_PHASE)

BATCH = 16
train_dl = DataLoader(train_ds, batch_size=BATCH, shuffle=True,  num_workers=2, pin_memory=True)
val_dl   = DataLoader(val_ds,   batch_size=BATCH, shuffle=False, num_workers=2, pin_memory=True)
test_dl  = DataLoader(test_ds,  batch_size=BATCH, shuffle=False, num_workers=2, pin_memory=True)
```

```
Embryos → train 562 | val 70 | test 70
100%                                                562/562 [16:30<00:00,  1.55s/it]
Kept 28,361 frames
100%                                                70/70 [02:05<00:00,  1.73s/it]
Kept 3,799 frames
100%                                                70/70 [01:59<00:00,  1.95s/it]
Kept 3,495 frames
```

🔍 Preview data-augmentations

```
# @title 🔍 Preview data-augmentations
import random, matplotlib.pyplot as plt
import torch

# ------------------------------------------------
# helper to de-normalise a 1×H×W tensor
def denorm(x):
    return (x*0.5 + 0.5).clamp(0,1).squeeze().cpu().numpy()

# 1)  Eight random augmented frames ----------------
N = 8
idxs = random.sample(range(len(train_ds)), N)
```

```
    fig, axes = plt.subplots(1, N, figsize=(2.2*N, 2.5))

    for ax, i in zip(axes, idxs):
        img, lbl = train_ds[i]
        ax.imshow(denorm(img), cmap='gray')
        ax.set_title(IDX2PHASE[lbl.item()], fontsize=7)
        ax.axis('off')
    fig.suptitle("Random augmented samples", fontsize=12)
    plt.show()

    # 2)  Same frame, 5 different augmentations --------
    idx = random.choice(range(len(train_ds)))
    base_path, _ = train_ds.samples[idx]
    base_img = Image.open(base_path).convert("L")

    fig, axes = plt.subplots(1, 5, figsize=(11, 2.4))
    for ax in axes:
        aug = train_tfm(base_img)
        ax.imshow(denorm(aug), cmap='gray')
        ax.axis('off')
    fig.suptitle("Five augmentations of the same frame", fontsize=12)
    plt.show()
```



🧠 Model

```
# @title 🧠 Model
from torchvision.models import efficientnet_b3, EfficientNet_B3_Weights
from torchvision.models import efficientnet_v2_s, EfficientNet_V2_S_Weights


n_classes = train_ds.num_classes
model = efficientnet_v2_s(weights=EfficientNet_V2_S_Weights.IMAGENET1K_V1)

model.features[0][0] = nn.Conv2d(
    1,
    model.features[0][0].out_channels,
    kernel_size=3, stride=2, padding=1, bias=False
)

# classifier head
model.classifier[1] = nn.Linear(
    model.classifier[1].in_features,
    n_classes
)

model.to(device)

# ----------------------------------------------------------------
# 3.  optimiser / loss
#    (unfreeze full network – you have fewer epochs, so lower LR)
optimizer = torch.optim.AdamW(
```

```
    model.parameters(),
    lr=1e-4,            # start lower; V2-S is deeper than B3
    weight_decay=1e-4
)
criterion = nn.CrossEntropyLoss()
```

```
Downloading: "https://download.pytorch.org/models/efficientnet_v2_s-dd5fe13b.pth" to /root/.cache/torch/hub/checkpoints/efficientne
100%|██████████| 82.7M/82.7M [00:03<00:00, 27.8MB/s]
```

🚂 Training Loop with Early Stopping

```
# @title 🚂 Training Loop with Early Stopping
best_loss = float("inf")
patience = 10
epochs_no_improve = 0
best_state = None

EPOCHS = 100
for epoch in range(1, EPOCHS+1):
    # Train
    model.train()
    tr_loss, tr_corr = 0., 0
    for x, y in tqdm(train_dl, desc=f"Epoch {epoch} train", leave=False):
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()
        logits = model(x)
        loss = criterion(logits, y)
        loss.backward()
        optimizer.step()
        tr_loss += loss.item()*x.size(0)
        tr_corr += (logits.argmax(1)==y).sum().item()
    tr_loss /= len(train_ds)
    tr_acc = tr_corr / len(train_ds)

    # Validate
    model.eval()
    val_loss, val_corr = 0., 0
    with torch.no_grad():
        for x, y in val_dl:
            x, y = x.to(device), y.to(device)
            logits = model(x)
            loss = criterion(logits, y)
            val_loss += loss.item()*x.size(0)
            val_corr += (logits.argmax(1)==y).sum().item()
    val_loss /= len(val_ds)
    val_acc = val_corr / len(val_ds)

    print(f"[{epoch:02d}] train loss {tr_loss:.4f} acc {tr_acc:.3f} | val loss {val_loss:.4f} acc {val_acc:.3f}")

    if val_loss < best_loss - 1e-6:
        best_loss = val_loss
        epochs_no_improve = 0
        best_state = {k: v.clone() for k, v in model.state_dict().items()}
    else:
        epochs_no_improve += 1
        if epochs_no_improve >= patience:
            print("🛑 Early stopping.")
            break

# restore best
model.load_state_dict(best_state)
```

```
[01] train loss 1.3463 acc 0.498 | val loss 1.1714 acc 0.572

[02] train loss 1.0700 acc 0.590 | val loss 1.2143 acc 0.563

[03] train loss 0.9804 acc 0.620 | val loss 1.1771 acc 0.582

[04] train loss 0.9191 acc 0.640 | val loss 1.1428 acc 0.585

[05] train loss 0.8657 acc 0.658 | val loss 1.1580 acc 0.588

[06] train loss 0.8174 acc 0.671 | val loss 1.1334 acc 0.603

[07] train loss 0.7819 acc 0.689 | val loss 1.1434 acc 0.600

[08] train loss 0.7479 acc 0.701 | val loss 1.1999 acc 0.609

[09] train loss 0.7073 acc 0.713 | val loss 1.3062 acc 0.599

[10] train loss 0.6765 acc 0.726 | val loss 1.2363 acc 0.601

[11] train loss 0.6425 acc 0.740 | val loss 1.2732 acc 0.599

[12] train loss 0.6052 acc 0.753 | val loss 1.2971 acc 0.597

[13] train loss 0.5853 acc 0.762 | val loss 1.3100 acc 0.599

[14] train loss 0.5524 acc 0.774 | val loss 1.4595 acc 0.596

[15] train loss 0.5258 acc 0.786 | val loss 1.4728 acc 0.590

[16] train loss 0.4921 acc 0.800 | val loss 1.4648 acc 0.597
🔴 Early stopping.
<All keys matched successfully>
```

📊 Test-Set Report

```
# @title 📊 Test-Set Report
model.eval()
preds, trues = [], []
with torch.no_grad():
    for x, y in tqdm(test_dl, desc="Test inference"):
        logits = model(x.to(device))
        preds.extend(logits.argmax(1).cpu().numpy())
        trues.extend(y.numpy())

# Get unique labels present in the test set
unique_labels = sorted(list(set(trues)))

# Filter and order target_names based on unique_labels
target_names = [IDX2PHASE[i] for i in sorted(set(trues))]


report = classification_report(trues, preds, labels=unique_labels, target_names=target_names, digits=3)
print(report)
cm = confusion_matrix(trues, preds, labels=unique_labels)
print("Confusion matrix\n", cm)
```

```
Test inference: 100%                                                219/219 [00:09<00:00, 22.78it/s]

              precision    recall  f1-score   support

        tPB2      0.723     0.862     0.786       275
        tPNa      0.835     0.771     0.801       301
        tPNf      0.794     0.820     0.806       305
          t2      0.793     0.793     0.793       319
          t3      0.508     0.521     0.514       121
          t4      0.720     0.529     0.610       306
          t5      0.306     0.338     0.322       201
          t6      0.344     0.405     0.372       215
          t7      0.338     0.455     0.388       200
          t8      0.598     0.396     0.477       270
         t9+      0.703     0.568     0.628       259
          tM      0.614     0.850     0.713       234
         tSB      0.659     0.665     0.662       206
          tB      0.581     0.637     0.608       146
         tEB      0.744     0.480     0.584       127
         tHB      0.000     0.000     0.000        10

    accuracy                          0.626      3495
   macro avg      0.579     0.568     0.567      3495
weighted avg      0.639     0.626     0.625      3495

Confusion matrix
 [[237  24  13   1   0   0   0   0   0   0   0   0   0   0   0   0]
 [ 38 232  29   2   0   0   0   0   0   0   0   0   0   0   0   0]
 [ 10  20 250  25   0   0   0   0   0   0   0   0   0   0   0   0]
 [  5   0  17 253  31   9   3   0   0   0   0   1   0   0   0   0]
 [  0   0   0  27  63  22   9   0   0   0   0   0   0   0   0   0]
 [  4   0   1   7  23 162  78  29   1   0   0   1   0   0   0   0]
 [  5   0   0   0   5  24  68  87  10   1   1   0   0   0   0   0]
 [  5   0   1   0   1   6  41  87  60   9   2   3   0   0   0   0]
 [  5   0   0   0   0   0  14  37  91  37  11   5   0   0   0   0]
 [  0   0   0   1   1   2   7  11  96 107  39   5   1   0   0   0]
 [  4   0   1   2   0   0   1   2  11  25 147  64   2   0   0   0]
 [  3   0   2   1   0   0   1   0   0   0   8 199  20   0   0   0]
 [  3   1   1   0   0   0   0   0   0   0   1  40 137  22   1   0]
```